# Mental Health Journal Application

A Comprehensive Digital Platform for Mood Tracking and Personal Reflection

**Rahma Abdulhameed - 7358**
**Salma Yousry - 7353**
**Merna Nader - 7608**
**Omar Mahmoud - 7441**

May 24, 2025

# Contents


**1 Introduction** **4**
- 1.1 Purpose . . . 4
- 1.2 Definitions . . . 4
- 1.3 Goals and Objectives . . . 5
  - 1.3.1 Primary Goals . . . 5
  - 1.3.2 Specific Objectives . . . 5
- 1.4 Project Plan . . . 6
  - 1.4.1 Phase 1: System Design and Architecture . . . 6
  - 1.4.2 Phase 2: Backend Development . . . 6
  - 1.4.3 Phase 3: Frontend Development . . . 6
  - 1.4.4 Phase 4: AI Integration and Enhancement . . . 6
  - 1.4.5 Phase 5: Testing and Quality Assurance . . . 6
  - 1.4.6 Phase 6: Deployment and Documentation . . . 6

**2 Requirements** **7**
- 2.1 User Requirements . . . 7
  - 2.1.1 Functional User Requirements . . . 7
  - 2.1.2 Non-Functional User Requirements . . . 7
- 2.2 System Requirements . . . 8
  - 2.2.1 Software Requirements . . . 8
  - 2.2.2 Technical System Requirements . . . 8

**3 Software Process** **9**
- 3.1 Type of Software Process . . . 9
- 3.2 Division of Process . . . 9
  - 3.2.1 Phase 1: Requirements Analysis and System Design . . . 9
  - 3.2.2 Phase 2: Backend Development . . . 9
  - 3.2.3 Phase 3: Frontend Development . . . 10
  - 3.2.4 Phase 4: Integration and System Testing . . . 10
  - 3.2.5 Phase 5: Deployment and Configuration . . . 10
  - 3.2.6 Phase 6: Documentation and Maintenance . . . 10

**4 Architectural Design** **11**
- 4.1 System Architecture . . . 11
- 4.2 Application Architecture . . . 12

**5 API Endpoint Testing with Postman** **14**
- 5.1 Authentication Tests . . . 14
- 5.2 Journal Entry Tests . . . 14
- 5.3 Music Recommendation Tests . . . 15
- 5.4 Postman Test Collection . . . 15
- 5.5 Error Case Testing . . . 16

**6 System Diagrams and Modeling** **17**
6.1 Use Case Diagram . . . . . 17
6.2 Class Diagram . . . . . 18
6.3 State Machine Diagrams . . . . . 19
6.4 Activity Diagrams . . . . . 20
6.5 Entity Relational Diagram . . . . . 21
6.6 Sequence Diagram . . . . . 22
6.7 Web Application Preview . . . . . 23

# 1 Introduction

## 1.1 Purpose

Mental health awareness has become increasingly important in today's fast-paced world, where individuals face various stressors and emotional challenges on a daily basis.

The Mental Health Journal application serves as a comprehensive digital solution designed to help users track, understand, and manage their emotional well-being through systematic journaling and mood monitoring.

The integration of artificial intelligence through Google's Gemini API adds an innovative dimension to traditional journaling by offering personalized song recommendations based on users' emotional states, creating a more engaging and therapeutic experience.

## 1.2 Definitions

- **Mental Health Journal:** A digital platform that allows users to record daily thoughts, emotions, and experiences in a structured format for self-reflection and emotional tracking.
- **Mood Tracking:** The systematic monitoring of emotional states over time.
- **FastAPI:** A modern, fast web framework for building APIs with Python, providing automatic interactive API documentation and high performance.
- **React:** A JavaScript library for building user interfaces.
- **Gemini API:** Google's generative artificial intelligence service that provides natural language processing capabilities for content generation and analysis.
- **JWT (JSON Web Tokens):** A compact, URL-safe means of representing claims to be transferred between two parties for secure authentication.
- **PostgreSQL:** An advanced open-source relational database system known for its reliability, feature robustness, and performance.
- **SQLAlchemy:** A Python SQL toolkit and Object-Relational Mapping (ORM) library that provides a full suite of well-known enterprise-level persistence patterns.

## 1.3 Goals and Objectives

### 1.3.1 Primary Goals

- **Mental Health Support:** Create an accessible platform that encourages regular self-reflection and emotional awareness.
- **Mood Pattern Recognition:** Enable users to track and visualize their emotional patterns over time.
- **Personalized Experience:** Provide AI-driven recommendations that adapt to user preferences and emotional states.
- **Task Management Integration:** Combine emotional well-being tracking with productivity tools.

### 1.3.2 Specific Objectives

- Develop a secure, user-friendly web application with authentication and authorization mechanisms.
- Implement a comprehensive mood tracking system with visual representations and color-coded indicators.
- Integrate AI-powered song recommendations based on mood analysis and journal content.
- Develop a todo list management system linked to daily entries for comprehensive life tracking.

## 1.4 Project Plan

The development of the Mental Health Journal application follows a systematic approach divided into several key phases:

### 1.4.1 Phase 1: System Design and Architecture

- Database schema design and entity relationship modeling
- API endpoint specification and documentation
- User interface wireframing

### 1.4.2 Phase 2: Backend Development

- Database implementation
- RESTful API development using FastAPI framework
- Integration with Google Gemini API

### 1.4.3 Phase 3: Frontend Development

- User interface implementation with responsive design
- Integration with backend APIs for data management

### 1.4.4 Phase 4: AI Integration and Enhancement

- Gemini API integration for natural language processing
- Personalization engine for user-specific recommendations

### 1.4.5 Phase 5: Testing and Quality Assurance

- Unit testing for individual components and functions
- Integration testing for API endpoints and database operations

### 1.4.6 Phase 6: Deployment and Documentation

- Production environment setup and configuration
- Database migration and initial data population

# 2 Requirements

## 2.1 User Requirements

These requirements are derived from use cases that capture the essential functionalities.

### 2.1.1 Functional User Requirements

1. **User Registration and Authentication**
   - Create new accounts with unique usernames and email addresses
   - Log in securely using credentials
   - Receive appropriate error messages for invalid login attempts
2. **Mood Tracking**
   - Select from predefined mood categories
   - Visual representations of moods through color-coded indicators over time
3. **Journal Entry Management**
   - Create daily journal entries with text notes and mood selection
   - View past entries
4. **Todo List Management**
   - Create todo items with descriptions and priority levels
   - Set deadlines for todo items and mark them complete
5. **AI-Powered Song Recommendations**
   - Receive song recommendations that consider both mood and journal content
   - Get mood-based recommendations without requiring a journal entry

### 2.1.2 Non-Functional User Requirements

1. **Usability**
   - User-friendly interface
   - Clear and provide actionable guidance
2. **Accessibility**
   - Responsive and adapt to different screen sizes
3. **Performance**
   - Respond to user interactions within 1 second
   - AI-powered recommendations generated within 10 seconds
4. **Security and Privacy**
   - User data shall be encrypted and stored securely
   - Personal information shall not be shared with third parties

## 2.2 System Requirements

System requirements define the technical specifications and constraints that the system must satisfy to function correctly and efficiently.

### 2.2.1 Software Requirements

**Backend System Requirements**

- **Database:** PostgreSQL 12.0 or higher
- **Web Server:** Uvicorn ASGI server
- **External APIs:** Access to Google Gemini API

**Frontend System Requirements**

- **Node.js:** Version 16.0 or higher for development
- **JavaScript:** ES6+ support required
- **CSS:** CSS3 support for modern styling features

**Development Environment Requirements**

- **Version Control:** Git for source code management
- **Database Management:** pgAdmin or similar PostgreSQL management tools

### 2.2.2 Technical System Requirements

**Database Requirements**

- **ACID Compliance:** Full ACID transaction support for data integrity
- **Security:** Encrypted connections and secure authentication

**API Requirements**

- **REST Architecture:** RESTful API design following HTTP standards
- **JSON Format:** Data exchange in JSON format
- **Authentication:** JWT-based authentication system

**Security Requirements**

- **HTTPS:** All communications must use HTTPS encryption
- **Password Security:** Bcrypt hashing for password storage
- **Session Management:** Secure JWT token handling

# 3 Software Process

## 3.1 Type of Software Process

The Mental Health Journal application development follows a **Hybrid Agile-Waterfall** software process model, specifically implementing an **Iterative and Incremental Development** approach.

#### Process Selection

The hybrid approach was chosen for several key reasons:

1. **Project Complexity:** The application involves multiple interconnected components (frontend, backend, database, AI integration) requiring careful coordination and planning.
2. **Risk Management:** The iterative nature allows for early identification and mitigation of technical risks, particularly with AI integration and database design.
3. **Quality Assurance:** Incremental development enables continuous testing and validation throughout the development cycle.

## 3.2 Division of Process

The software development process is divided into 6 major phases, each containing multiple iterations and specific deliverables.

### 3.2.1 Phase 1: Requirements Analysis and System Design

- Use case analysis and user story development
- System architecture design and technology selection
- Database schema design and entity relationship modeling
- API specification and endpoint definition

### 3.2.2 Phase 2: Backend Development

- Database setup and connection configuration
- SQLAlchemy models and relationships implementation
- Google Gemini API integration
- Song recommendation service implementation
- Mood analysis and content processing

#### 3.2.3 Phase 3: Frontend Development

- React application setup and routing
- Journal entry creation and editing interfaces
- Mood tracking and selection components
- Entry viewing and management functionality
- Todo list management interface
- Song recommendation display and integration

#### 3.2.4 Phase 4: Integration and System Testing

- End-to-end system integration testing
- AI feature validation and accuracy testing

#### 3.2.5 Phase 5: Deployment and Configuration

- Production environment setup and configuration
- Database deployment and data migration

#### 3.2.6 Phase 6: Documentation and Maintenance

- Technical documentation completion
- API documentation finalization
- Developer documentation and code comments

# 4 Architectural Design

## 4.1 System Architecture

The system architecture consists of multiple layers and components working together to provide a cohesive user experience:

### Architectural Layers

1. **Presentation Layer (Frontend)**
   - Responsibility: User interface rendering and user interaction handling
   - Communication: RESTful API calls to the backend application layer
2. **Application Layer (Backend)**
   - Responsibility: Business logic implementation, API endpoint management, and request processing
   - Communication: Database queries through ORM and external API integration
3. **Data Layer (Database)**
   - Responsibility: Persistent data storage and retrieval
   - Communication: SQL queries through SQLAlchemy ORM
4. **External Services Layer**
   - Responsibility: AI-powered content generation and analysis
   - Communication: HTTPS REST API calls to external services

### System Components Interaction

The system components interact through well-defined interfaces and protocols:

- **Frontend & Backend:** RESTful API over HTTPS with JSON data exchange
- **Backend & Database:** SQLAlchemy ORM with connection pooling
- **Backend & External APIs:** HTTP/HTTPS requests with authentication

## 4.2 Application Architecture

The application architecture defines the internal structure and organization of components within each layer of the system.

### Backend Application Architecture

The backend follows a layered architecture pattern with clear separation of concerns:

#### Service Layer

- External API integration and communication
- Data transformation and processing

#### Data Access Layer

- Database schema definition and relationships
- Query optimization and data retrieval

#### Configuration Layer

- Database connection configuration
- API keys and external service configuration

### Frontend Application Architecture

The frontend implements a component-based architecture using React patterns:

#### State Management Architecture

- **Local State:** Component-level state using React hooks (useState, useEffect)
- **Server State:** API data management with custom hooks and caching

#### Routing Architecture

- **Lazy Loading:** Code splitting for improved performance

### Data Flow Architecture

The application implements a unidirectional data flow pattern:

#### Request Flow

1. User interaction triggers event in React component
2. Component makes API call to backend endpoint
3. Backend controller validates request and calls service layer
4. Service layer implements logic and calls data access layer
5. Data access layer performs database operations
6. Response flows back through layers to frontend

#### Authentication Flow

1. User submits login credentials
2. Frontend sends authentication request to backend
3. Backend validates credentials and generates JWT token
4. Token is stored in frontend local storage
5. Backend validates token and processes authenticated requests

#### AI Integration Flow

1. Frontend sends request with entry/mood data to backend
2. Backend service extracts mood and journal content
3. Service creates contextual prompt for Gemini API
4. External API call to Google Gemini service
5. Recommendation returned to frontend for display

# 5 API Endpoint Testing with Postman

## 5.1 Authentication Tests

- **Test User Registration**
  - **Request**:

```
POST /users/register
Body:
{
    "username": "newuser",
    "email": "new@example.com",
    "password": "SecurePass123"
}
```

  - **Validations**:
    * Status Code: 200 (Success) or 400 (Bad Request for duplicates)
    * Response contains username and email (without password)
    * Verify duplicate username/email returns 400
- **Test User Login**
  - **Request**:

```
POST /token
Body (form-data):
username: existinguser
password: CorrectPass123
```

  - **Validations**:
    * Status Code: 200 (Success) or 401 (Unauthorized)
    * Response contains access_token
    * Verify invalid credentials return 401

## 5.2 Journal Entry Tests

- **Test Entry Creation with Valid Todo**
  - **Request**:

```
POST /users/1/entries/
Headers:
Authorization: Bearer <token>
Body:
{
    "entry_date": "2023-11-15",
    "notes": "Test entry",
    "title": "Test Title",
```

```
    "mood_id": 1
}
```

- **Validations**:
  * Status Code: 201 (Created)
  * Response contains entry_id
  * Verify required fields validation

## 5.3 Music Recommendation Tests

- **Test Song Recommendation by Mood**
  - **Request**:

```
POST /song-recommendation/by-mood
Headers:
Authorization: Bearer <token>
Body:
{
    "mood_name": "happy"
}
```

  - **Validations**:
    * Status Code: 200
    * Response contains song_recommendation
    * Verify invalid mood returns proper error
- **Test Entry-Specific Recommendation**
  - **Request**:

```
GET /users/1/entries/5/song-recommendation
Headers:
Authorization: Bearer <token>
```

  - **Validations**:
    * Status Code: 200 (or 404 if entry doesn't exist)
    * Response contains either recommendation or error message

## 5.4 Postman Test Collection

- **Collection Structure**:
  1. Authentication
     - Register New User
     - Login with Valid Credentials
     - Login with Invalid Credentials

2. Journal Entries
   - Create New Entry
   - Get Entries by Date Range
   - Create Entry with Todos
3. Music Recommendations
   - Get Recommendation by Mood
   - Get Recommendation for Entry
   - Invalid Mood Test

- **Environment Variables**:

```
{
    "base_url": "http://localhost:8000",
    "auth_token": "{{login_response.access_token}}"
}
```

## 5.5 Error Case Testing

- **Unauthorized Access**:
  - Call protected endpoints without token
  - Verify 401 status code
- **Invalid Todo Creation**:
  - Attempt to create todo without description
  - Verify proper validation error (422)
- **Non-Existent Entry**:
  - Request song recommendation for invalid entry ID
  - Verify 404 status code

Table 1: Postman Test Matrix

| **Endpoint** | **Test Case** | **Expected Status** | **Validation** |
|---|---|---|---|
| POST /register | Valid registration | 200 | User data in response |
| POST /register | Duplicate username | 400 | Error message |
| POST /token | Valid login | 200 | Token in response |
| POST /entries | Missing required field | 422 | Validation details |
| GET /song-recommendation | Invalid mood | 400 | Error message |

# 6 System Diagrams and Modeling

## 6.1 Use Case Diagram

The use case diagram provides a high-level view of the Mental Health Journal system's functionality from the user's perspective. It identifies the key actors, their interactions with the system, and the relationships between different use cases.

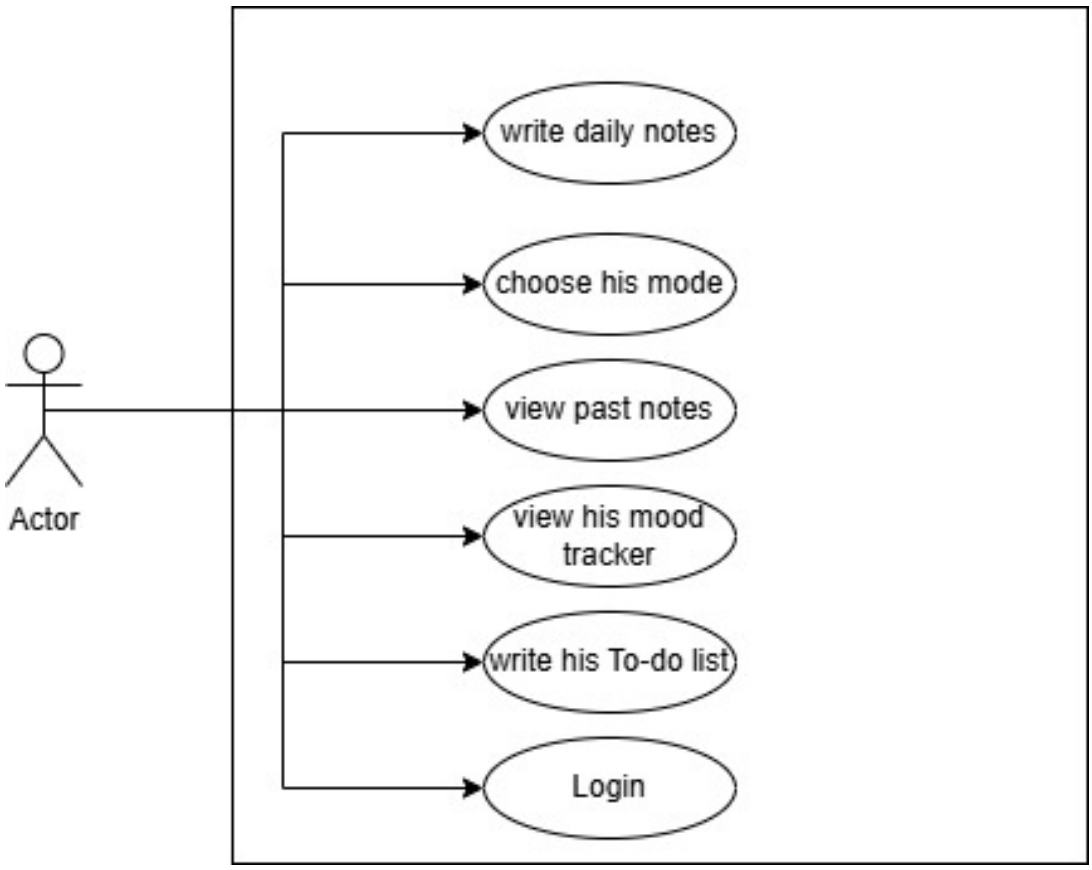


Figure 1: Use Case Diagram for Mental Health Journal System

## 6.2 Class Diagram

The class diagram provides a detailed view of the Mental Health Journal system's object-oriented structure, illustrating the relationships between classes, their attributes, methods, and the overall system architecture.

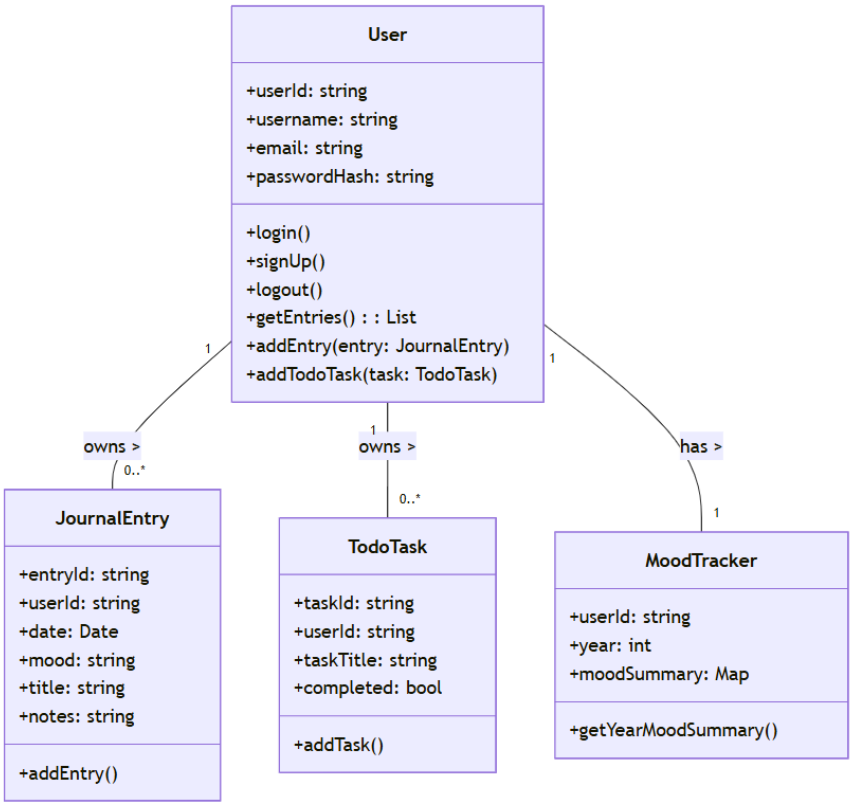


Figure 2: Class Diagram for Mental Health Journal System

## 6.3 State Machine Diagrams

State machine diagrams provide a dynamic view of the Mental Health Journal system by modeling the behavior of system components over time. These diagrams illustrate how different entities in the system transition between various states in response to events, user actions, and system conditions.

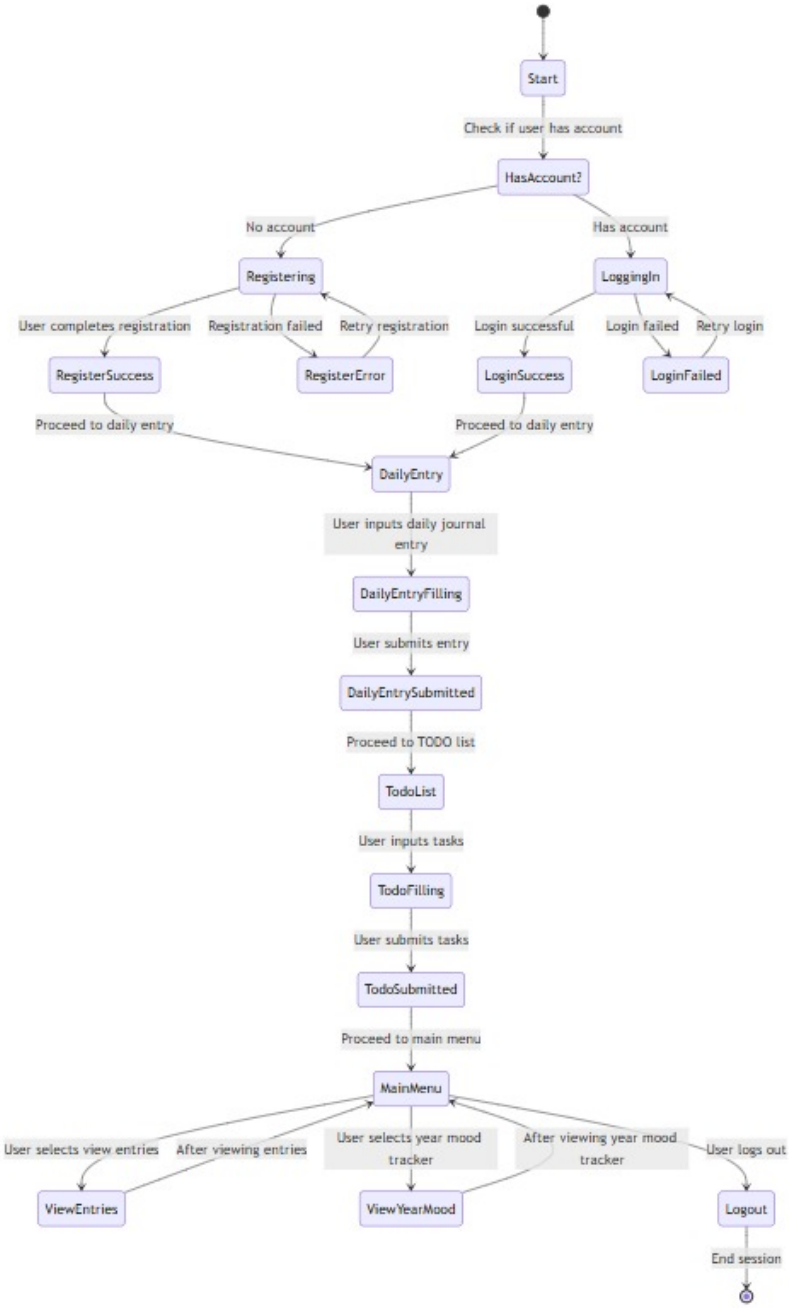


Figure 3: State Machine Diagrams for Mental Health Journal System

## 6.4 Activity Diagrams

Activity diagrams provide a detailed view of the flow of events and processes within the Mental Health Journal system. These diagrams illustrate the sequential and parallel activities that occur during the execution of key use cases, showing how different system components interact to accomplish user goals.

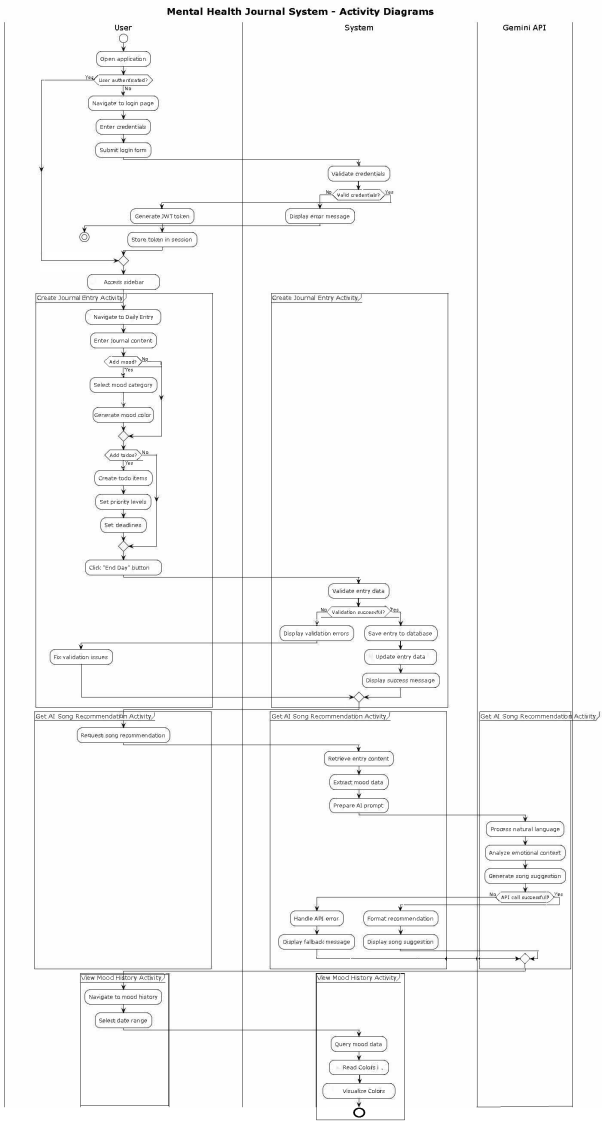


Figure 4: Activity Diagram for Mental Health Journal System Use Cases

## 6.5 Entity Relational Diagram

The Entity-Relationship Diagram (ERD) illustrates the key entities in the system, such as users, journal entries, and activities, as well as the relationships between them.

The ERD helps visualize how data is structured and how various components of the database interact with each other to support the system's functionality.

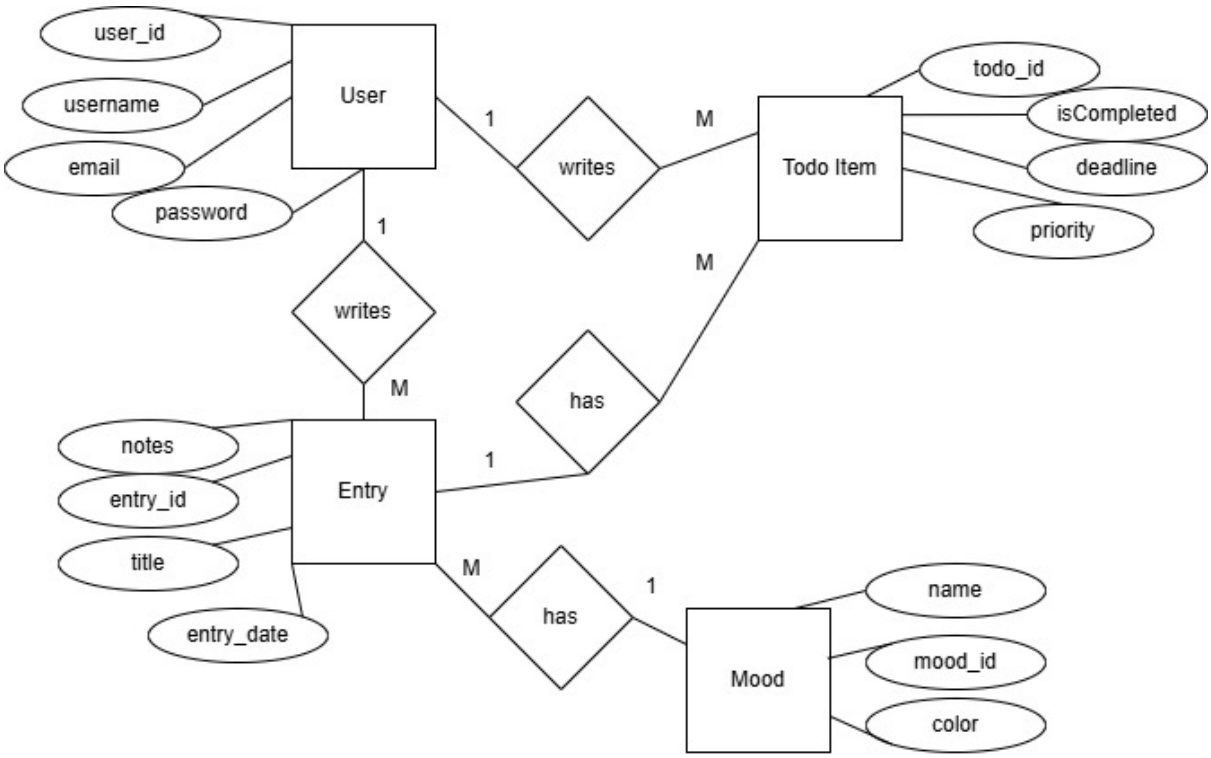


Figure 5: Entity-Relationship Diagram for Mental Health Journal System Use Cases

## 6.6 Sequence Diagram

The sequence diagram offers a dynamic view of the Mental Health Journal system. It illustrates the sequence of method calls, messages exchanged, and the flow of control during specific use cases. This diagram highlights how different components collaborate to perform operations, showing the chronological order of events and the lifelines of participating objects within the system.

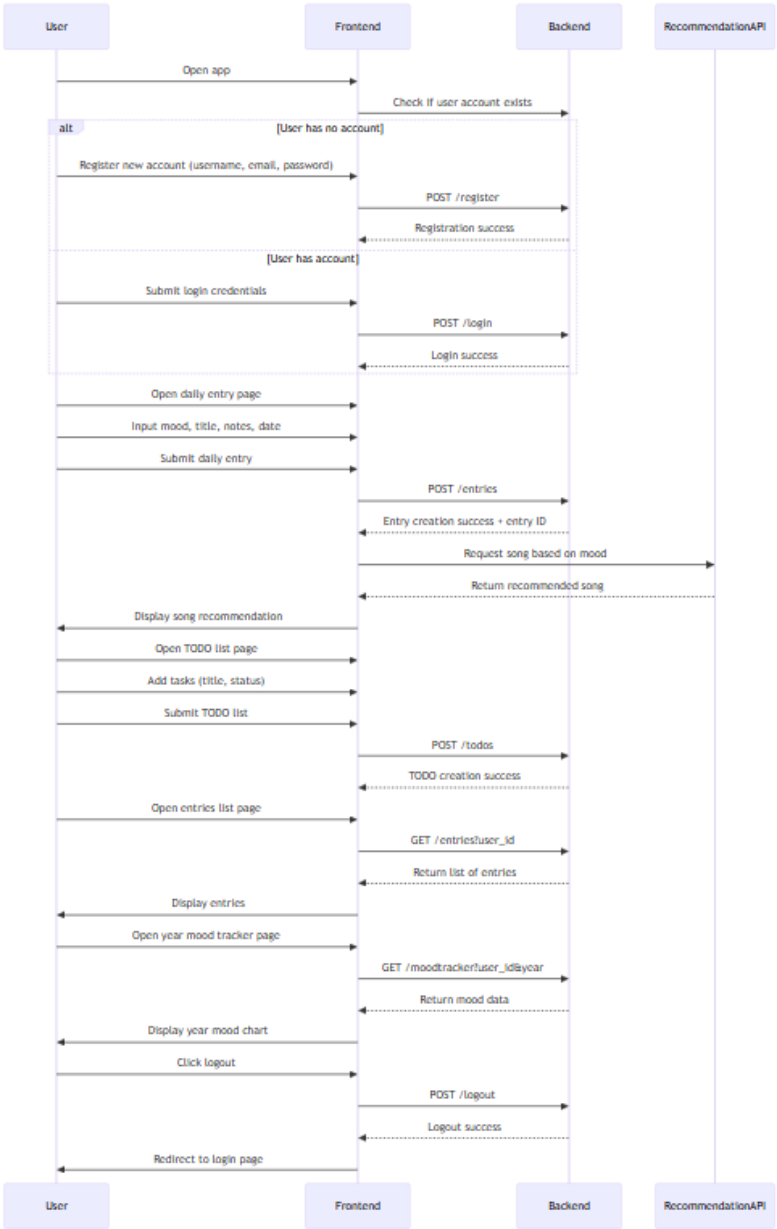


Figure 6: Sequence Diagram for Mental Health Journal System Use Cases

## 6.7 Web Application Preview

The following images showcase a preview from the functional Mental Health Journal web application. These preview shots provide a visual overview of the system's user interface and demonstrate key features.

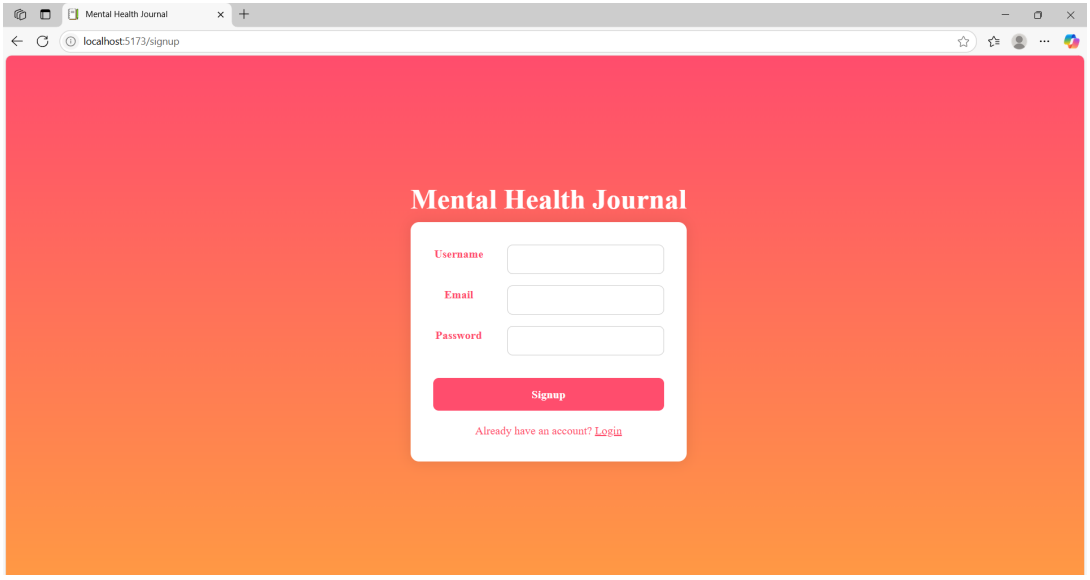


Figure 7: Signup Page of the Mental Health Journal

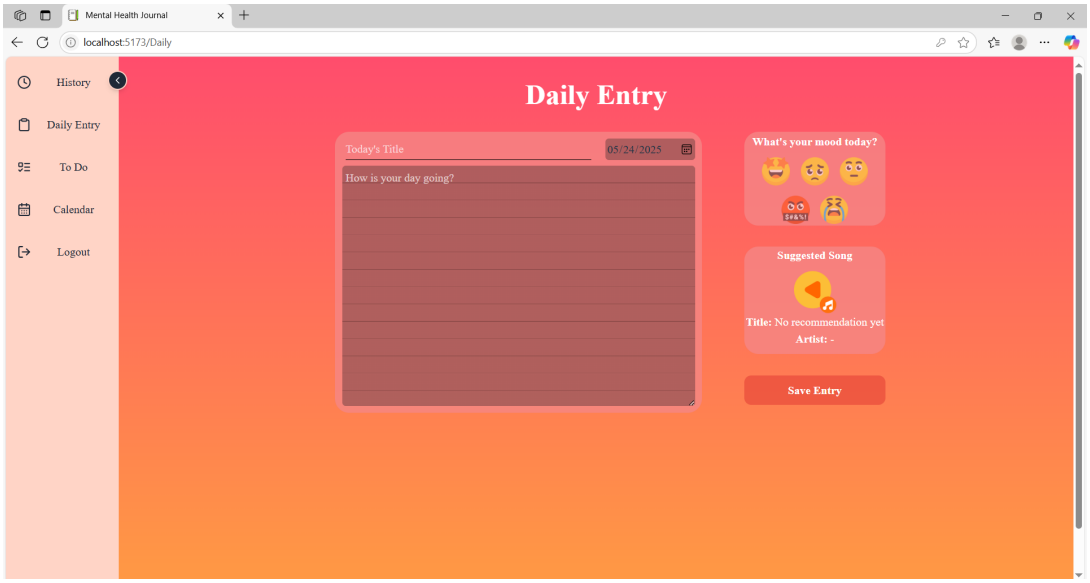


Figure 8: Daily Entry Page of the Mental Health Journal

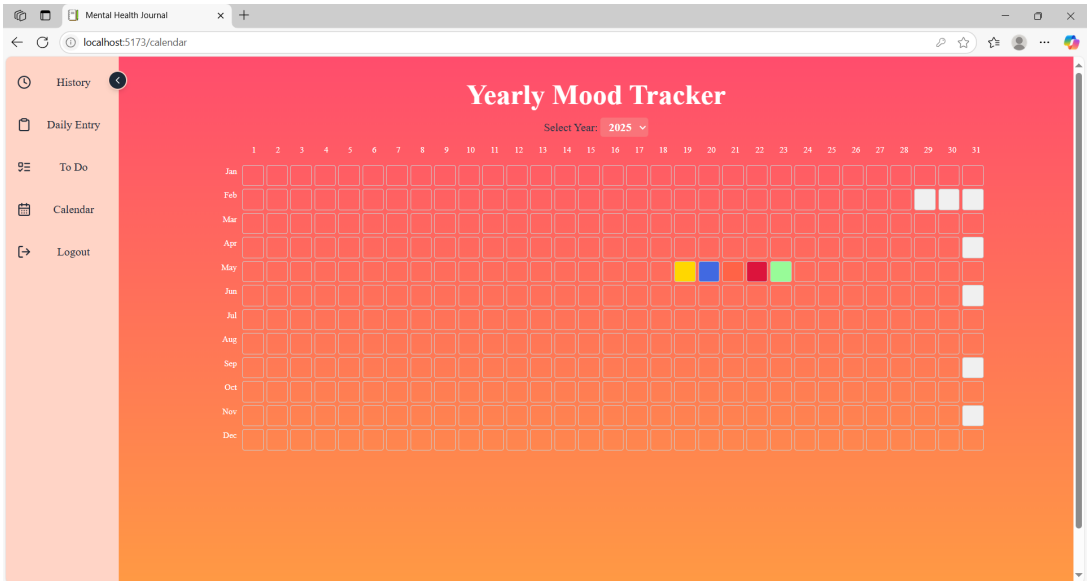


Figure 9: Calendar Page of the Mental Health Journal

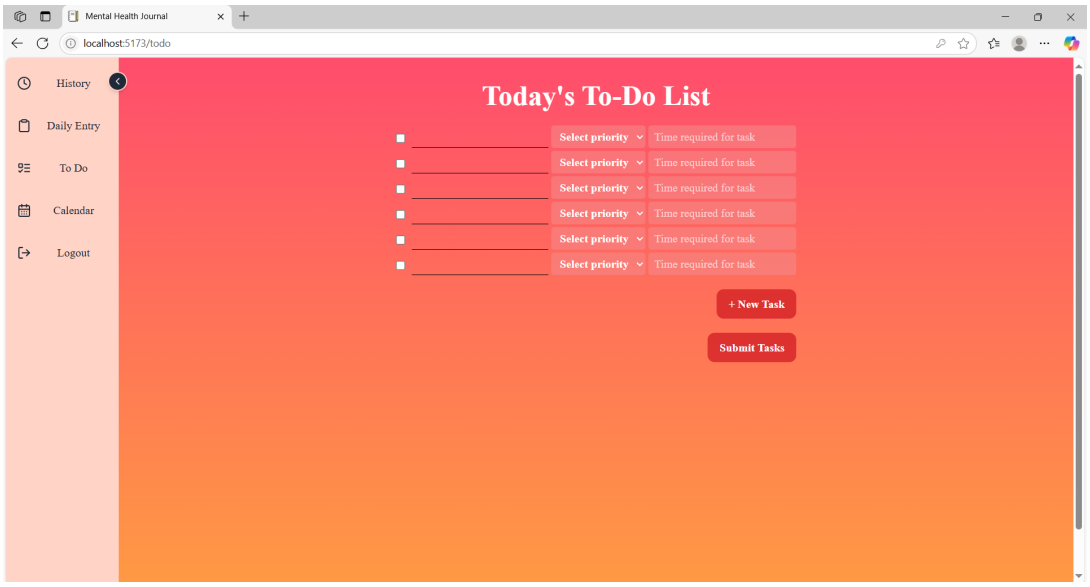


Figure 10: To-Do List Page of the Mental Health Journal